

Student Management System (Flutter UI Project)

Chapter 1

Introduction

The **Student Management System** is a mobile application developed using the **Flutter framework** and **Dart programming language**. The application is designed to help users manage student information efficiently through a simple and user-friendly interface. It allows users to add, view, search, edit, and delete student records while storing data locally using the SQLite database.

The project demonstrates the practical implementation of Flutter development, SQLite database integration, and Object-Oriented Programming (OOP) concepts such as classes, objects, constructors, encapsulation, navigation, and CRUD (Create, Read, Update, Delete) operations.

Project Objective

The objectives of this project are:

- Develop a functional Flutter mobile application.
- Digitally manage student information.
- Perform CRUD (Create, Read, Update, Delete) operations.
- Reduce manual record-keeping.
- Store student information using SQLite.
- Learn Flutter application development.
- Improve Object-Oriented Programming skills.
- Gain practical experience in mobile application development and documentation.

Problem Statement

Many schools, colleges, and training institutes still maintain student information manually. Managing records on paper is time-consuming and increases the risk of data loss and human errors. Teachers and administrators need a simple digital system to store, update, and retrieve student information quickly.

The Student Management System provides an efficient solution by allowing users to manage student records through an easy-to-use mobile application with local database support.

Chapter 2

Feasibility Study

Technical Feasibility

The project is technically feasible because all required technologies are freely available.

- Flutter Framework
- Dart Programming Language
- SQLite Database
- Visual Studio Code
- Android Emulator / Android Device
- Windows Operating System

No paid software or special hardware is required.

Economic Feasibility

The project has almost no development cost.

Development Cost

- Flutter SDK – Free
- Dart SDK – Free
- SQLite – Free
- Visual Studio Code – Free
- Windows PC – Already Available

Therefore, the project is economically feasible.

Operational Feasibility

The application is simple and easy to operate. Users can easily:

- Add Student
- View Student List
- Search Student
- Edit Student Information
- Delete Student
- View Student Details

No technical knowledge is required.

Requirement Analysis

Functional Requirements

The system provides the following features:

- Home Screen
- Add Student
- View Student List
- Search Student
- Student Details
- Edit Student Information
- Delete Student
- Local SQLite Database
- Automatic List Refresh

Non-functional Requirements

Security

- Input validation prevents incorrect information.

Performance

- Fast data retrieval using SQLite.

Reliability

- Student records are stored securely in the local database.

Usability

- Simple and user-friendly mobile interface.

Chapter 3

System Design

System Overview

The Student Management System allows users to manage student information through a mobile application.

The system performs the following operations:

- Store student information
- View student records
- Search students
- Update student information
- Delete student records

System Flowchart

Start



Home Screen



Student List



| Add Student |

| Search Student |

| View Details |

| Edit Student |

| Delete Student |



Perform Selected Task



SQLite Database



Updated Student List



Exit

Database Design

Table Name: Students

Field	Type
id	INTEGER (Primary Key)
name	TEXT
studentId	TEXT
department	TEXT
semester	TEXT
phone	TEXT
email	TEXT

Data Flow Diagram (DFD)

User

|



Student Management System

|

|— Add Student

|— Update Student

|— Delete Student

|— Search Student

|— View Student

|



SQLite Database

|



Display Result

Tools & Technologies Used

Item	Name
Programming Language	Dart
Framework	Flutter
Database	SQLite

IDE Visual Studio Code

Operating System Windows 10/11

Version Control Git & GitHub

System Implementation and UI Design

The project was implemented using Flutter as a mobile application.

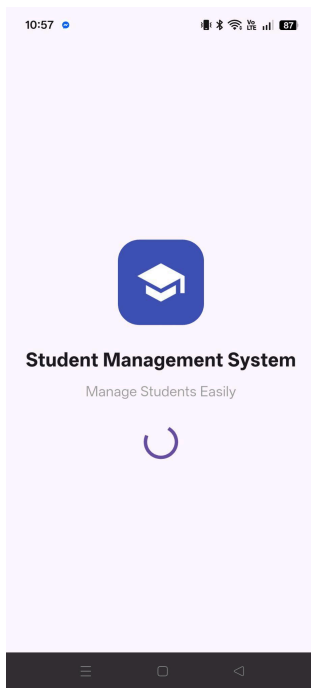
The program uses:

- Flutter Widgets
- Material Design
- Classes
- Objects
- Constructors
- Encapsulation
- SQLite Database
- CRUD Operations
- Navigation
- Forms
- Validation
- ListView
- Search Functionality

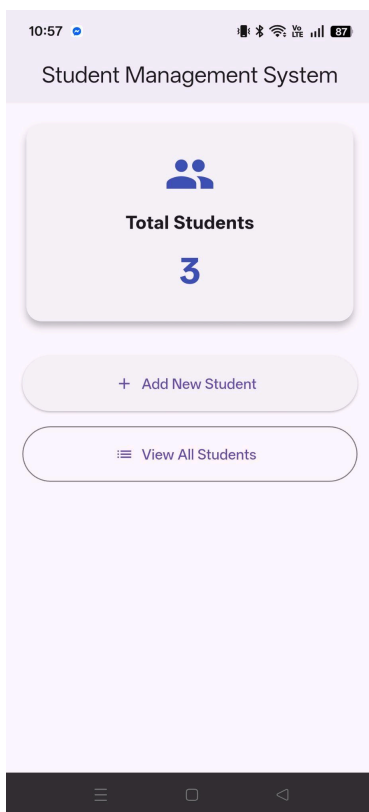
The application allows users to perform all operations interactively through a clean and responsive graphical user interface.

Screenshots to Include

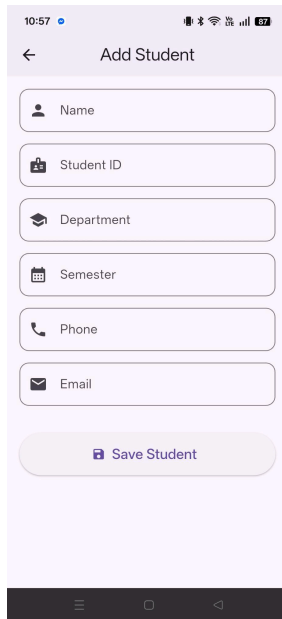
- **Splash Screen**



- **Home Screen**



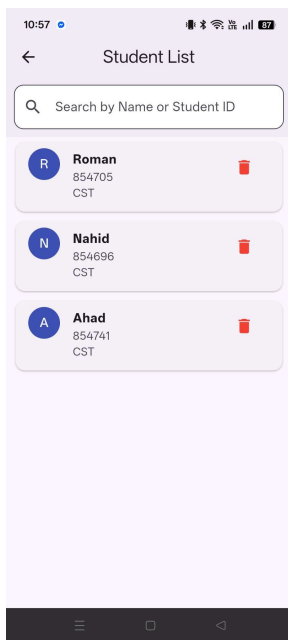
- **Add Student Screen**



A mobile app screen titled "Add Student" with a back arrow. It features six input fields with icons: Name (person), Student ID (ID card), Department (graduation cap), Semester (calendar), Phone (phone), and Email (envelope). A "Save Student" button with a save icon is at the bottom.

Field	Icon
Name	Person
Student ID	ID Card
Department	Graduation Cap
Semester	Calendar
Phone	Phone
Email	Envelope

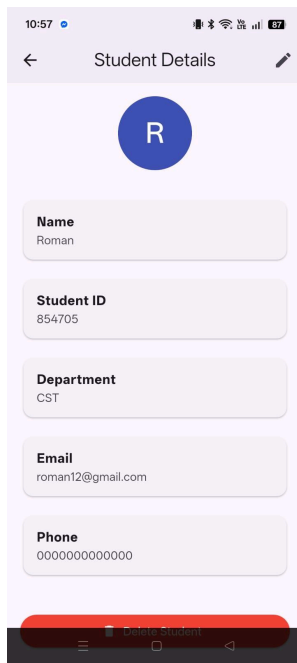
- **Student List Screen**



A mobile app screen titled "Student List" with a back arrow. It has a search bar "Search by Name or Student ID". Below is a list of three students, each with a circular initial icon, name, ID, department, and a delete icon.

Initial	Name	ID	Department	Action
R	Roman	854705	CST	Delete
N	Nahid	854696	CST	Delete
A	Ahad	854741	CST	Delete

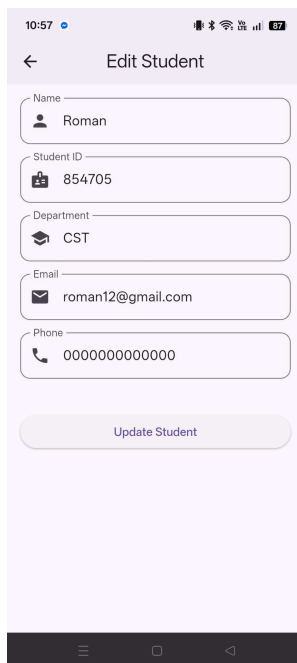
- **Student Details Screen**



A mobile app screen titled "Student Details". At the top is a status bar with the time 10:57 and various icons. Below the title bar is a blue circular profile picture with the letter "R". The screen contains five data fields, each with a label and a value: "Name" (Roman), "Student ID" (854705), "Department" (CST), "Email" (roman12@gmail.com), and "Phone" (000000000000). At the bottom is a red button labeled "Delete Student".

Field	Value
Name	Roman
Student ID	854705
Department	CST
Email	roman12@gmail.com
Phone	000000000000

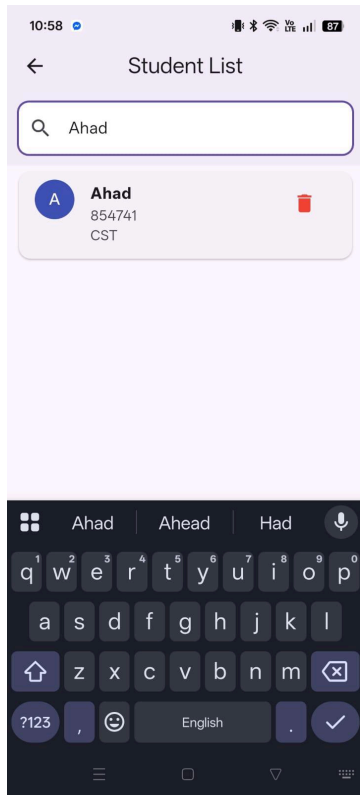
-
- **Edit Student Screen**



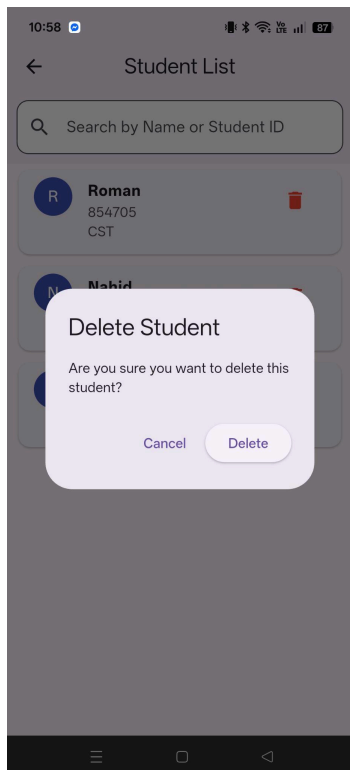
A mobile app screen titled "Edit Student". At the top is a status bar with the time 10:57 and various icons. Below the title bar is a back arrow. The screen contains five data fields, each with a label, an icon, and a value: "Name" (Roman) with a person icon, "Student ID" (854705) with a ID card icon, "Department" (CST) with a graduation cap icon, "Email" (roman12@gmail.com) with an envelope icon, and "Phone" (000000000000) with a phone icon. At the bottom is a button labeled "Update Student".

Field	Value
Name	Roman
Student ID	854705
Department	CST
Email	roman12@gmail.com
Phone	000000000000

- **Search Student Screen**



- **Delete Confirmation Dialog**



Chapter 4

Challenges Faced

Several challenges were encountered during development:

- Designing a responsive user interface.
- Connecting Flutter with SQLite.
- Implementing CRUD operations.
- Managing navigation between screens.
- Refreshing data after update or deletion.
- Validating user input.
- Debugging runtime and database errors.

These challenges were solved through continuous testing, debugging, and improving the application logic.

Future Improvements

The following features can be added in future versions:

- User Login & Authentication
- Firebase Cloud Database
- Student Profile Picture Upload
- Attendance Management
- Result Management System
- PDF Report Generation
- Dark Mode
- Push Notifications
- Cloud Backup & Synchronization
- AI-based Student Performance Analysis

Conclusion

The **Student Management System** successfully provides an efficient way to manage student information through a Flutter mobile application. It demonstrates the practical implementation of Flutter, Dart, SQLite, and Object-Oriented Programming concepts. The system reduces manual work, improves data accuracy, and offers a simple and user-friendly interface for managing student records. Future enhancements such as cloud integration, authentication, attendance tracking, and AI-powered analytics can further improve the application's functionality and usability.

